composite_full_test.m — source listing

Shri Vishal, Department of Aerospace Engineering, Karunya Institute of Technology and Sciences.

Note: this listing contains the code as supplied. The file ends inside `animate_compressive` (inside the `for frame` loop, line 1286), and the local helper functions (`define_material_library`, `make_stress_strain_curve`, `make_compressive_curve`, `build_report_lines`, `build_validation_lines`, `style_ax`, `make_anim_fig`, `make_dark_ax`, `compare_samples`) were not included.

```
function composite_full_test()
% ========================================================================
% COMPOSITE MATERIAL FULL TEST SUITE - WITH VIBRATION TESTING
%
% Tests: Tensile (ASTM D3039), Drop-Weight Impact (ASTM D7136),
%        Charpy Impact (ASTM E23), Izod Impact (ASTM D256),
%        Flexural / 3-Point Bend (ASTM D790), Compressive (ASTM D695),
%        Vibration (ASTM D3580)
%
% Features:
%   * User inputs specimen dimensions before each test
%   * Material properties remain constant from library
%   * Dynamic report shows selected test type
%   * All animations work properly with user-defined dimensions
% ========================================================================
clc; clear all; close all;
rng(42);

% ========================================================================
%  STARTUP DIALOG — Tester Name & Company
% ========================================================================
dlg_ans = inputdlg( ...
    {'Tester Name:', 'Company / Lab Name:', 'Report Reference No:'}, ...
    'Test Setup Information', 1, ...
    {'Eng. ---', 'Your Company Ltd.', 'REF-2026-001'});

if isempty(dlg_ans)
    tester_name   = 'Eng. ---';
    company_name  = 'Lab ---';
    ref_no        = 'REF-###';
else
    tester_name   = strtrim(dlg_ans{1});
    company_name  = strtrim(dlg_ans{2});
    ref_no        = strtrim(dlg_ans{3});
end

data = define_material_library();

% ========================================================================
%  MAIN FIGURE
% ========================================================================
scr = get(0,'ScreenSize');
fw  = min(1440, scr(3)-20);
fh  = min(920,  scr(4)-50);
fig = figure('Name','Composite Material Full Test Suite', ...
    'NumberTitle','off', ...
    'Position',[10 25 fw fh], ...
    'Color',[0.92 0.92 0.94], ...
    'MenuBar','none','ToolBar','none','Resize','on', ...
    'CloseRequestFcn',@(~,~)closereq_cb());

    function closereq_cb()
        try; delete(fig); catch; end
    end

% ---- Header ----
uicontrol('Style','text', ...
    'String', sprintf('COMPOSITE MATERIAL TEST SUITE  —  %s  |  %s', company_name, tester_name), ...
    'Parent',fig,'Units','normalized','Position',[0 0.956 1 0.044], ...
    'BackgroundColor',[0.04 0.14 0.36],'ForegroundColor','white', ...
    'FontSize',12,'FontWeight','bold','HorizontalAlignment','center');

uicontrol('Style','text', ...
    'String','Tensile (D3039) | Drop-Weight (D7136) | Charpy (E23) | Izod (D256) | Flexural (D790) | Compressive (D695) | Vibration (D3580) | Date: 4/16/2026', ...
    'Parent',fig,'Units','normalized','Position',[0 0.924 1 0.032], ...
    'BackgroundColor',[0.10 0.24 0.52],'ForegroundColor',[0.85 0.92 1.0], ...
    'FontSize',8.0,'HorizontalAlignment','center');

% ---- Row 1 Controls: Material + Test Type ----
uicontrol('Style','text','String','Material:', ...
    'Parent',fig,'Units','normalized','Position',[0.005 0.894 0.058 0.026], ...
    'BackgroundColor',[0.92 0.92 0.94],'FontSize',8.5,'FontWeight','bold', ...
    'HorizontalAlignment','left');

mat_names = cell(1, length(data));
for mi = 1:length(data)
    mat_names{mi} = sprintf('[%d] %s', data(mi).test_no, data(mi).sample_name);
end
sample_popup = uicontrol('Style','popupmenu','String',mat_names, ...
    'Parent',fig,'Units','normalized','Position',[0.062 0.895 0.155 0.025], ...
```

```
    'FontSize',8.5,'BackgroundColor','white','Callback',@on_sample_change);

uicontrol('Style','text','String','Test Type:', ...
    'Parent',fig,'Units','normalized','Position',[0.222 0.894 0.055 0.026], ...
    'BackgroundColor',[0.92 0.92 0.94],'FontSize',8.5,'FontWeight','bold', ...
    'HorizontalAlignment','left');

test_popup = uicontrol('Style','popupmenu', ...
    'String',{'Tensile (ASTM D3039)','Drop-Weight Impact (D7136)', ...
              'Charpy Impact (ASTM E23)','Izod Impact (ASTM D256)', ...
              'Flexural / 3-Pt Bend (D790)','Compressive (ASTM D695)', ...
              'Vibration (ASTM D3580)'}, ...
    'Parent',fig,'Units','normalized','Position',[0.278 0.895 0.165 0.025], ...
    'FontSize',8.5,'BackgroundColor','white');

% ---- Row 1 Buttons ----
btn_colors = {[0.06 0.44 0.16],[0.60 0.12 0.06],[0.22 0.06 0.52],[0.42 0.28 0.04],[0.04 0.36 0.48]};
btn_labels = {'▶  GENERATE REPORT','▼  RUN ANIMATION','≡  COMPARE ALL','?  MAT. INFO','✎  EDIT TESTER'};
btn_cbs    = {@(~,~)generate_report_callback, @run_animation, @compare_samples, @show_material_info, @edit_tester};
for bi = 1:5
    uicontrol('Style','pushbutton','String',btn_labels{bi}, ...
        'Parent',fig,'Units','normalized', ...
        'Position',[0.447+(bi-1)*0.108 0.894 0.104 0.026], ...
        'BackgroundColor',btn_colors{bi},'ForegroundColor','white', ...
        'FontSize',8.5,'FontWeight','bold','Callback',btn_cbs{bi});
end

% ---- Status bar ----
status_lbl = uicontrol('Style','text', ...
    'String',sprintf('  Ready  |  Tester: %s  |  Company: %s  |  Ref: %s', tester_name, company_name, ref_no), ...
    'Parent',fig,'Units','normalized','Position',[0 0 1 0.023], ...
    'BackgroundColor',[0.10 0.10 0.13],'ForegroundColor',[0.28 1.0 0.44], ...
    'FontSize',8.0,'HorizontalAlignment','left');

% ---- Left Panel: Report ----
uicontrol('Style','text','String','  TEST REPORT  (Machine Format)', ...
    'Parent',fig,'Units','normalized','Position',[0.002 0.865 0.310 0.026], ...
    'BackgroundColor',[0.04 0.14 0.36],'ForegroundColor','white', ...
    'FontSize',8.5,'FontWeight','bold','HorizontalAlignment','left');

report_box = uicontrol('Style','listbox','String',{}, ...
    'Parent',fig,'Units','normalized','Position',[0.002 0.024 0.310 0.840], ...
    'BackgroundColor',[0.98 0.99 1.0],'ForegroundColor',[0.04 0.04 0.10], ...
    'FontSize',7.8,'FontName','Courier New','Enable','inactive');

% ---- Center-Top: Load vs Displacement ----
uicontrol('Style','text','String','  LOAD  vs  DISPLACEMENT', ...
    'Parent',fig,'Units','normalized','Position',[0.316 0.865 0.345 0.026], ...
    'BackgroundColor',[0.04 0.14 0.36],'ForegroundColor','white', ...
    'FontSize',8.5,'FontWeight','bold','HorizontalAlignment','left');

ax_ld = axes('Parent',fig,'Units','normalized', ...
    'Position',[0.335 0.490 0.310 0.360], ...
    'Color','white','Box','on','GridColor',[0.70 0.70 0.70]);

% ---- Center-Bottom: Stress-Strain ----
uicontrol('Style','text','String','  STRESS  vs  STRAIN', ...
    'Parent',fig,'Units','normalized','Position',[0.316 0.430 0.345 0.026], ...
    'BackgroundColor',[0.04 0.14 0.36],'ForegroundColor','white', ...
    'FontSize',8.5,'FontWeight','bold','HorizontalAlignment','left');

ax_ss = axes('Parent',fig,'Units','normalized', ...
    'Position',[0.335 0.060 0.310 0.350], ...
    'Color','white','Box','on','GridColor',[0.70 0.70 0.70]);

% ---- Right-Top: Secondary Plot ----
uicontrol('Style','text','String','  FREQUENCY RESPONSE', ...
    'Parent',fig,'Units','normalized','Position',[0.665 0.865 0.330 0.026], ...
    'BackgroundColor',[0.04 0.38 0.14],'ForegroundColor','white', ...
    'FontSize',8.5,'FontWeight','bold','HorizontalAlignment','left');

ax_en = axes('Parent',fig,'Units','normalized', ...
    'Position',[0.682 0.490 0.305 0.360], ...
    'Color','white','Box','on','GridColor',[0.70 0.70 0.70]);

% ---- Right-Bottom: Validation ----
uicontrol('Style','text','String','  VALIDATION  —  Computed  vs  Reported', ...
    'Parent',fig,'Units','normalized','Position',[0.665 0.430 0.330 0.026], ...
    'BackgroundColor',[0.04 0.38 0.14],'ForegroundColor','white', ...
    'FontSize',8.5,'FontWeight','bold','HorizontalAlignment','left');

val_box = uicontrol('Style','listbox','String',{}, ...
    'Parent',fig,'Units','normalized','Position',[0.665 0.024 0.330 0.405], ...
    'BackgroundColor',[0.97 1.0 0.97],'ForegroundColor',[0.02 0.18 0.04], ...
    'FontSize',7.6,'FontName','Courier New','Enable','inactive');

% ---- App State ----
app.data          = data;
app.sample_popup  = sample_popup;
app.test_popup    = test_popup;
app.report_box    = report_box;
```

```
app.val_box       = val_box;
app.ax_ld         = ax_ld;
app.ax_ss         = ax_ss;
app.ax_en         = ax_en;
app.status_lbl    = status_lbl;
app.tester_name   = tester_name;
app.company_name  = company_name;
app.ref_no        = ref_no;
app.current_test_type = 'Tensile (ASTM D3039)';
guidata(fig, app);

on_sample_change([], []);

% ========================================================================
%  GENERATE REPORT CALLBACK
% ========================================================================
    function generate_report_callback(~,~)
        app2 = guidata(fig);
        idx = get(app2.sample_popup, 'Value');
        d = app2.data(idx);
        test_type_idx = get(app2.test_popup, 'Value');
        test_types = {'Tensile (ASTM D3039)', 'Drop-Weight Impact (D7136)', ...
                      'Charpy Impact (ASTM E23)', 'Izod Impact (ASTM D256)', ...
                      'Flexural / 3-Pt Bend (D790)', 'Compressive (ASTM D695)', ...
                      'Vibration (ASTM D3580)'};
        current_test = test_types{test_type_idx};
        set(app2.report_box, 'String', build_report_lines(d, app2, current_test));
        set(app2.status_lbl, 'String', sprintf('  Report generated for %s - %s', d.sample_name, current_test));
    end

% ========================================================================
%  EDIT TESTER INFO
% ========================================================================
    function edit_tester(~,~)
        app2 = guidata(fig);
        ans2 = inputdlg({'Tester Name:','Company / Lab Name:','Reference No:'}, ...
            'Edit Tester Info',1, ...
            {app2.tester_name, app2.company_name, app2.ref_no});
        if ~isempty(ans2)
            app2.tester_name  = strtrim(ans2{1});
            app2.company_name = strtrim(ans2{2});
            app2.ref_no       = strtrim(ans2{3});
            guidata(fig, app2);
            on_sample_change([],[]);
        end
    end

% ========================================================================
%  CALLBACK: sample / test type changed
% ========================================================================
    function on_sample_change(~, ~)
        app2 = guidata(fig);
        if ~ishandle(fig); return; end
        idx = get(app2.sample_popup, 'Value');
        d   = app2.data(idx);
        test_type_idx = get(app2.test_popup, 'Value');
        test_types = {'Tensile (ASTM D3039)', 'Drop-Weight Impact (D7136)', ...
                      'Charpy Impact (ASTM E23)', 'Izod Impact (ASTM D256)', ...
                      'Flexural / 3-Pt Bend (D790)', 'Compressive (ASTM D695)', ...
                      'Vibration (ASTM D3580)'};
        current_test = test_types{test_type_idx};
        [sv, tv] = make_stress_strain_curve(d);
        set(app2.report_box, 'String', build_report_lines(d, app2, current_test));
        do_plot_ld(app2.ax_ld, d, sv, tv);
        do_plot_ss(app2.ax_ss, d, sv, tv);
        do_plot_frequency(app2.ax_en, d);
        set(app2.val_box, 'String', build_validation_lines(d));
        set(app2.status_lbl,'String', sprintf( ...
            '  [%s] UTS=%.1f MPa | UCS=%.1f MPa | E=%.1f GPa | Natural Freq=%.1f Hz | Tester: %s | %s | Test: %s', ...
            d.sample_name, d.uts, d.ucs, d.young_modulus/1e3, d.natural_freq, ...
            app2.tester_name, app2.company_name, current_test));
        app2.current_test_type = current_test;
        guidata(fig, app2);
    end

% ========================================================================
%  MATERIAL INFO
% ========================================================================
    function show_material_info(~,~)
        app2 = guidata(fig);
        idx  = get(app2.sample_popup,'Value');
        d    = app2.data(idx);
        msg  = sprintf([ ...
            '=== MATERIAL: %s ===\n\n' ...
            'Company       : %s\n' ...
            'Tested By     : %s\n' ...
            'Reference     : %s\n\n' ...
            'Class         : %s\n' ...
            'Fibre/Matrix  : %s\n\n' ...
            'Young Modulus : %.2f GPa\n' ...
            'UTS (Tensile) : %.1f MPa\n' ...
```

```
            'UCS (Compress): %.1f MPa\n' ...
            'Yield Str.    : %.1f MPa\n' ...
            'Break Str.    : %.1f MPa\n' ...
            'Elongation    : %.2f %%\n' ...
            'Density       : %.0f kg/m³\n' ...
            'Poisson Ratio : %.3f\n' ...
            'Natural Freq  : %.1f Hz\n' ...
            'Damping Ratio : %.3f\n\n' ...
            'Default Specimen Dimensions:\n' ...
            '  Width      = %.2f mm\n' ...
            '  Thickness  = %.2f mm\n' ...
            '  Gauge Len  = %.2f mm\n' ...
            '  Xsect Area = %.3f mm²\n\n' ...
            'NOTE: You can enter custom dimensions before each test!'], ...
            d.sample_name, app2.company_name, app2.tester_name, app2.ref_no, ...
            d.material_class, d.fibre_matrix, ...
            d.young_modulus/1e3, d.uts, d.ucs, d.yield_strength, d.break_strength, ...
            d.elongation, d.density, d.poisson, d.natural_freq, d.damping_ratio, ...
            d.width, d.thickness, d.gauge_length, d.area);
        msgbox(msg, sprintf('Material Info — %s', d.sample_name),'help');
    end

% ========================================================================
%  GET USER DIMENSIONS
% ========================================================================
    function dims = get_user_dimensions(d, test_type)
        default_width = d.width;
        default_thickness = d.thickness;
        default_gauge = d.gauge_length;

        if contains(test_type, 'Tensile')
            prompt = {sprintf('Specimen Width (mm) [default: %.2f]:', default_width), ...
                      sprintf('Specimen Thickness (mm) [default: %.2f]:', default_thickness), ...
                      sprintf('Gauge Length (mm) [default: %.2f]:', default_gauge)};
            dlg_title = 'Enter Tensile Specimen Dimensions';
            dims = inputdlg(prompt, dlg_title, 1, ...
                {num2str(default_width), num2str(default_thickness), num2str(default_gauge)});
        elseif contains(test_type, 'Compressive')
            prompt = {sprintf('Specimen Width (mm) [default: %.2f]:', default_width), ...
                      sprintf('Specimen Thickness (mm) [default: %.2f]:', default_thickness), ...
                      sprintf('Specimen Height (mm) [default: %.2f]:', default_gauge*1.2)};
            dlg_title = 'Enter Compressive Specimen Dimensions';
            dims = inputdlg(prompt, dlg_title, 1, ...
                {num2str(default_width), num2str(default_thickness), num2str(default_gauge*1.2)});
        elseif contains(test_type, 'Vibration')
            prompt = {sprintf('Specimen Width (mm) [default: %.2f]:', default_width), ...
                      sprintf('Specimen Thickness (mm) [default: %.2f]:', default_thickness), ...
                      sprintf('Specimen Length (mm) [default: %.2f]:', default_gauge*1.5)};
            dlg_title = 'Enter Vibration Specimen Dimensions';
            dims = inputdlg(prompt, dlg_title, 1, ...
                {num2str(default_width), num2str(default_thickness), num2str(default_gauge*1.5)});
        elseif contains(test_type, 'Impact') || contains(test_type, 'Charpy') || contains(test_type, 'Izod')
            prompt = {sprintf('Specimen Width (mm) [default: %.2f]:', default_width), ...
                      sprintf('Specimen Thickness (mm) [default: %.2f]:', default_thickness)};
            dlg_title = 'Enter Impact Specimen Dimensions';
            dims = inputdlg(prompt, dlg_title, 1, ...
                {num2str(default_width), num2str(default_thickness)});
            dims{3} = num2str(80);
        elseif contains(test_type, 'Flexural')
            prompt = {sprintf('Specimen Width (mm) [default: %.2f]:', default_width), ...
                      sprintf('Specimen Thickness (mm) [default: %.2f]:', default_thickness), ...
                      'Support Span Length (mm) [default: 100]:'};
            dlg_title = 'Enter Flexural Specimen Dimensions';
            dims = inputdlg(prompt, dlg_title, 1, ...
                {num2str(default_width), num2str(default_thickness), '100'});
        else
            dims = {num2str(default_width), num2str(default_thickness), num2str(default_gauge)};
        end

        if isempty(dims)
            dims = [default_width, default_thickness, default_gauge];
        else
            for i = 1:length(dims)
                if isempty(str2double(dims{i})) || isnan(str2double(dims{i}))
                    if i == 1, dims{i} = default_width;
                    elseif i == 2, dims{i} = default_thickness;
                    else, dims{i} = default_gauge;
                    end
                else
                    dims{i} = str2double(dims{i});
                end
            end
        end
    end

% ========================================================================
%  RUN ANIMATION (dispatcher with user dimensions)
% ========================================================================
    function run_animation(~,~)
        app2 = guidata(fig);
        tsel = get(app2.test_popup,'Value');
```

```
        idx  = get(app2.sample_popup,'Value');
        d    = app2.data(idx);
        test_types = {'Tensile (ASTM D3039)','Drop-Weight Impact (D7136)', ...
                      'Charpy Impact (ASTM E23)','Izod Impact (ASTM D256)', ...
                      'Flexural / 3-Pt Bend (D790)','Compressive (ASTM D695)', ...
                      'Vibration (ASTM D3580)'};
        current_test = test_types{tsel};

        dims = get_user_dimensions(d, current_test);

        d_user = d;
        d_user.width = dims{1};
        d_user.thickness = dims{2};
        if length(dims) >= 3
            d_user.gauge_length = dims{3};
        end
        d_user.area = d_user.width * d_user.thickness;

        d_user.ult_load = d_user.uts * d_user.area / 1e3;
        d_user.yield_load = d_user.yield_strength * d_user.area / 1e3;
        d_user.break_load = d_user.break_strength * d_user.area / 1e3;

        set(app2.status_lbl, 'String', sprintf('  Running %s on %s (%.2f x %.2f mm) - User Dimensions', ...
            current_test, d.sample_name, d_user.width, d_user.thickness));
        drawnow;

        switch tsel
            case 1; animate_tensile(d_user, app2);
            case 2; animate_drop_impact(d_user, app2);
            case 3; animate_charpy(d_user, app2);
            case 4; animate_izod(d_user, app2);
            case 5; animate_flexural(d_user, app2);
            case 6; animate_compressive(d_user, app2);
            case 7; animate_vibration(d_user, app2);
        end
    end

% ========================================================================
%  PLOT: Load vs Displacement
% ========================================================================
    function do_plot_ld(ax, d, sv, tv)
        cla(ax); hold(ax,'on');
        dv = sv .* d.gauge_length / 100;
        lv = tv .* d.area / 1e3;
        fill(ax,[dv(1),dv,dv(end)],[0,lv,0],[0.07 0.35 0.75],'FaceAlpha',0.09,'EdgeColor','none');
        plot(ax,dv,lv,'Color',[0.07 0.35 0.75],'LineWidth',2.2);
        pk_d = d.peak_disp; pk_l = d.ult_load;
        br_d = d.max_disp;  br_l = d.break_load;
        plot(ax,pk_d,pk_l,'r^','MarkerSize',9,'LineWidth',2,'MarkerFaceColor',[1 0.25 0.08]);
        text(ax,pk_d+pk_d*0.04,pk_l*1.04,sprintf('ULT=%.2fkN',pk_l),'FontSize',7.5,'Color',[0.75 0.06 0.04],'FontWeight','bold');
        plot(ax,br_d,br_l,'kx','MarkerSize',10,'LineWidth',2.5);
        text(ax,br_d+pk_d*0.04,br_l*0.93,sprintf('BRK=%.2fkN',br_l),'FontSize',7.5,'Color',[0.18 0.18 0.18]);
        xlim(ax,[0,br_d*1.18]); ylim(ax,[0,pk_l*1.20]);
        style_ax(ax,'Displacement (mm)','Load (kN)', ...
            sprintf('L vs D  [%s | Test %d | %.2fx%.2f mm]',d.sample_name,d.test_no,d.width,d.thickness));
        hold(ax,'off');
    end

% ========================================================================
%  PLOT: Stress vs Strain
% ========================================================================
    function do_plot_ss(ax, d, sv, tv)
        cla(ax); hold(ax,'on');
        fill(ax,[sv(1),sv,sv(end)],[0,tv,0],[0.70 0.22 0.04],'FaceAlpha',0.10,'EdgeColor','none');
        plot(ax,sv,tv,'Color',[0.70 0.22 0.04],'LineWidth',2.2);
        el = linspace(0,sv(end)*0.08,40);
        sl = d.young_modulus * el / 100;
        plot(ax,el,sl,'--','Color',[0.20 0.58 0.24],'LineWidth',1.4);
        [~,iu] = max(tv);
        plot(ax,sv(iu),tv(iu),'r^','MarkerSize',9,'LineWidth',2,'MarkerFaceColor',[1 0.25 0.08]);
        text(ax,sv(iu)*1.02,tv(iu)*0.94,sprintf('UTS\n%.0fMPa',d.uts),'FontSize',7,'Color',[0.80 0.06 0.04]);
        xlim(ax,[0,sv(end)*1.12]); ylim(ax,[0,d.uts*1.20]);
        style_ax(ax,'Strain (%)','Stress (MPa)',sprintf('Stress-Strain  [%s | %s]',d.sample_name,d.material_class));
        hold(ax,'off');
    end

% ========================================================================
%  PLOT: Frequency Response (for vibration test)
% ========================================================================
    function do_plot_frequency(ax, d)
        cla(ax); hold(ax,'on');

        % Frequency range
        f = linspace(0.1, 500, 1000);

        % Transfer function (amplitude ratio)
        fn = d.natural_freq;
        zeta = d.damping_ratio;

        H = 1 ./ sqrt((1 - (f/fn).^2).^2 + (2*zeta*f/fn).^2);

```

```
        plot(ax, f, H, 'b-', 'LineWidth', 2.5);
        fill(ax, [f, fliplr(f)], [H, zeros(1, length(H))], [0.2 0.5 0.8], 'FaceAlpha', 0.3);

        % Mark natural frequency
        line(ax, [fn fn], [0 max(H)*1.1], 'Color', 'r', 'LineStyle', '--', 'LineWidth', 2);
        text(ax, fn*1.05, max(H)*0.9, sprintf('f_n = %.1f Hz', fn), 'FontSize', 8, 'Color', 'r', 'FontWeight', 'bold');

        xlim(ax, [0, min(500, fn*3)]);
        ylim(ax, [0, max(H)*1.2]);
        xlabel(ax, 'Frequency (Hz)', 'FontSize', 8.5);
        ylabel(ax, 'Amplitude Ratio', 'FontSize', 8.5);
        title(ax, sprintf('Frequency Response - %s', d.sample_name), 'FontSize', 9, 'FontWeight', 'bold');
        grid(ax, 'on');
        hold(ax, 'off');
    end

% ========================================================================
%  ANIMATION 1: TENSILE TEST
% ========================================================================
    function animate_tensile(d, app2)
        [sv, tv] = make_stress_strain_curve(d);

        afig = figure('Name', sprintf('Tensile Test ASTM D3039 — %s | %s | %s (%.2fx%.2f mm)', ...
            d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness), ...
            'NumberTitle', 'off', 'Position', [50 50 1300 800], ...
            'Color', [0.06 0.08 0.13], 'MenuBar', 'none', 'ToolBar', 'none');

        main_panel = uipanel('Parent', afig, 'Units', 'normalized', ...
            'Position', [0.02 0.05 0.96 0.90], 'BackgroundColor', [0.06 0.08 0.13], ...
            'BorderType', 'none');

        ax_spec = axes('Parent', main_panel, 'Units', 'normalized', ...
            'Position', [0.03 0.10 0.40 0.80], 'Color', [0.08 0.10 0.15]);
        axis(ax_spec, 'off'); hold(ax_spec, 'on');
        xlim(ax_spec, [0 1]); ylim(ax_spec, [0 1]);

        ax_ss_live = axes('Parent', main_panel, 'Units', 'normalized', ...
            'Position', [0.48 0.55 0.48 0.38], 'Color', [0.09 0.11 0.18]);
        xlabel(ax_ss_live, 'Strain (%)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        ylabel(ax_ss_live, 'Stress (MPa)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        title(ax_ss_live, 'Stress-Strain Curve', 'Color', 'white', 'FontSize', 10, 'FontWeight', 'bold');
        grid(ax_ss_live, 'on'); hold(ax_ss_live, 'on');

        ax_ld_live = axes('Parent', main_panel, 'Units', 'normalized', ...
            'Position', [0.48 0.08 0.48 0.38], 'Color', [0.09 0.11 0.18]);
        xlabel(ax_ld_live, 'Displacement (mm)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        ylabel(ax_ld_live, 'Load (kN)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        title(ax_ld_live, 'Load vs Displacement', 'Color', 'white', 'FontSize', 10, 'FontWeight', 'bold');
        grid(ax_ld_live, 'on'); hold(ax_ld_live, 'on');

        h_ss = plot(ax_ss_live, NaN, NaN, 'r-', 'LineWidth', 2.5);
        h_ld = plot(ax_ld_live, NaN, NaN, 'b-', 'LineWidth', 2.5);

        status_text = uicontrol('Parent', afig, 'Style', 'text', ...
            'String', sprintf('Testing %s (%.2f x %.2f mm)...', d.sample_name, d.width, d.thickness), ...
            'Position', [10, 10, 1280, 30], ...
            'BackgroundColor', [0.1 0.1 0.15], 'ForegroundColor', [0.28 1.0 0.44], ...
            'FontSize', 10, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');

        dv = sv .* d.gauge_length / 100;
        lv = tv .* d.area / 1e3;

        spec_left = 0.30;
        spec_right = 0.70;
        spec_bottom = 0.25;
        spec_top = 0.75;
        spec_width_norm = spec_right - spec_left;
        spec_height_norm = spec_top - spec_bottom;

        top_grip_y = spec_top + 0.05;
        bottom_grip_y = spec_bottom - 0.05;

        n_frames = 150;
        fracture_frame = round(n_frames * 0.85);

        acc_strain = [];
        acc_stress = [];
        acc_disp = [];
        acc_load = [];

        for frame = 1:n_frames
            if ~ishandle(afig); return; end

            progress = frame / n_frames;
            idx = min(length(sv), max(1, round(progress * length(sv))));

            current_strain = sv(idx);
            current_stress = tv(idx);
            current_disp = dv(idx);
            current_load = lv(idx);

```

```
            acc_strain = [acc_strain, current_strain];
            acc_stress = [acc_stress, current_stress];
            acc_disp = [acc_disp, current_disp];
            acc_load = [acc_load, current_load];

            set(h_ss, 'XData', acc_strain, 'YData', acc_stress);
            set(h_ld, 'XData', acc_disp, 'YData', acc_load);

            if max(acc_strain) > 0
                xlim(ax_ss_live, [0, max(acc_strain) * 1.1]);
                ylim(ax_ss_live, [0, max(acc_stress) * 1.1]);
            end
            if max(acc_disp) > 0
                xlim(ax_ld_live, [0, max(acc_disp) * 1.1]);
                ylim(ax_ld_live, [0, max(acc_load) * 1.1]);
            end

            cla(ax_spec);
            axis(ax_spec, 'off');
            hold(ax_spec, 'on');
            xlim(ax_spec, [0 1]); ylim(ax_spec, [0 1]);

            fill(ax_spec, [spec_left-0.08, spec_right+0.08, spec_right+0.08, spec_left-0.08], ...
                [top_grip_y, top_grip_y, top_grip_y+0.08, top_grip_y+0.08], ...
                [0.35, 0.35, 0.45], 'EdgeColor', [0.5, 0.5, 0.6], 'LineWidth', 2);

            fill(ax_spec, [spec_left-0.08, spec_right+0.08, spec_right+0.08, spec_left-0.08], ...
                [bottom_grip_y-0.08, bottom_grip_y-0.08, bottom_grip_y, bottom_grip_y], ...
                [0.35, 0.35, 0.45], 'EdgeColor', [0.5, 0.5, 0.6], 'LineWidth', 2);

            elongation_factor = 1 + progress * 0.15;

            if progress > 0.4
                necking_factor = 1 - (progress - 0.4) / 0.6 * 0.3 * (current_stress / d.uts);
                necking_factor = max(0.7, min(1, necking_factor));
            else
                necking_factor = 1;
            end

            stress_ratio = current_stress / d.uts;
            if stress_ratio < 0.5
                spec_color = [0.2, 0.4, 0.8];
            elseif stress_ratio < 0.8
                spec_color = [0.8, 0.6, 0.2];
            else
                spec_color = [0.9, 0.2, 0.1];
            end

            x_coords = [spec_left, ...
                        spec_left + spec_width_norm * (1-necking_factor)/2, ...
                        spec_left + spec_width_norm * (1-necking_factor)/2, ...
                        spec_right - spec_width_norm * (1-necking_factor)/2, ...
                        spec_right - spec_width_norm * (1-necking_factor)/2, ...
                        spec_right, spec_right, spec_left];

            current_spec_top = spec_top + (elongation_factor - 1) * spec_height_norm / 2;
            current_spec_bottom = spec_bottom - (elongation_factor - 1) * spec_height_norm / 2;

            y_coords = [current_spec_top, current_spec_top, ...
                        current_spec_bottom, current_spec_bottom, ...
                        current_spec_top, current_spec_top, ...
                        current_spec_bottom, current_spec_bottom];

            fill(ax_spec, x_coords, y_coords, spec_color, ...
                'EdgeColor', spec_color * 0.7, 'LineWidth', 1.5);

            if frame > fracture_frame && progress > 0.85
                crack_y = (current_spec_top + current_spec_bottom) / 2;
                for c = 1:8
                    crack_x = spec_left + spec_width_norm/2 + (rand() - 0.5) * spec_width_norm * 0.4;
                    crack_angle = (rand() - 0.5) * 60;
                    crack_length = 0.03 + rand() * 0.05;
                    dx = crack_length * cosd(crack_angle);
                    dy = crack_length * sind(crack_angle);
                    plot(ax_spec, [crack_x, crack_x + dx], [crack_y, crack_y + dy], ...
                        'y-', 'LineWidth', 1.5 + rand());
                end
            end

            text(ax_spec, 0.5, spec_top + 0.12, '↑ TENSILE LOAD ↑', ...
                'Color', [0.9, 0.9, 0.5], 'FontSize', 9, ...
                'HorizontalAlignment', 'center', 'FontWeight', 'bold');
            text(ax_spec, 0.5, spec_bottom - 0.08, '↓ TENSILE LOAD ↓', ...
                'Color', [0.9, 0.9, 0.5], 'FontSize', 9, ...
                'HorizontalAlignment', 'center', 'FontWeight', 'bold');

            text(ax_spec, 0.5, (current_spec_top + current_spec_bottom)/2, ...
                sprintf('%s\n%s\n%.1fx%.1f mm', d.sample_name, d.material_class, d.width, d.thickness), ...
                'Color', 'white', 'FontSize', 7, 'HorizontalAlignment', 'center', ...
                'FontWeight', 'bold', 'BackgroundColor', [0, 0, 0, 0.5]);

```

```
627             if frame <= fracture_frame
628                 status_str = sprintf('TENSILE | Frame %d/%d | Strain=%.2f%% | Stress=%.1fMPa | Load=%.3fkN | Dim: %.2fx%.2f mm', ...
629                     frame, n_frames, current_strain, current_stress, current_load, d.width, d.thickness);
630             else
631                 status_str = sprintf('TENSILE | Frame %d/%d | Strain=%.2f%% | Stress=%.1fMPa | Load=%.3fkN | FRACTURED | Dim: %.2fx%.2f mm', ...
632                     frame, n_frames, current_strain, current_stress, current_load, d.width, d.thickness);
633             end
634             set(status_text, 'String', status_str);
635             pause(0.03);
636         end
637 
638         if ishandle(afig)
639             text(ax_spec, 0.5, 0.5, ...
640                 sprintf('TENSILE TEST COMPLETE\nUTS = %.1f MPa\nMax Load = %.2f kN\nElongation = %.2f %%\nDimensions: %.2f x %.2f mm', ...
641                 d.uts, d.ult_load, d.elongation, d.width, d.thickness), ...
642                 'Color', [0.35, 1, 0.45], 'FontSize', 12, 'FontWeight', 'bold', ...
643                 'HorizontalAlignment', 'center', 'BackgroundColor', [0.04, 0.08, 0.12], ...
644                 'EdgeColor', [0.2, 0.55, 0.3], 'Margin', 10);
645         end
646     end
647 
648 % ========================================================================
649 %  ANIMATION 7: VIBRATION TEST (ASTM D3580)
650 % ========================================================================
651     function animate_vibration(d, app2)
652         afig = figure('Name', sprintf('Vibration Test ASTM D3580 — %s | %s | %s (%.2fx%.2f mm)', ...
653             d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness), ...
654             'NumberTitle', 'off', 'Position', [50 50 1300 800], ...
655             'Color', [0.06 0.08 0.13], 'MenuBar', 'none', 'ToolBar', 'none');
656 
657         main_panel = uipanel('Parent', afig, 'Units', 'normalized', ...
658             'Position', [0.02 0.05 0.96 0.90], 'BackgroundColor', [0.06 0.08 0.13], ...
659             'BorderType', 'none');
660 
661         ax_spec = axes('Parent', main_panel, 'Units', 'normalized', ...
662             'Position', [0.03 0.10 0.40 0.80], 'Color', [0.08 0.10 0.15]);
663         axis(ax_spec, 'off'); hold(ax_spec, 'on');
664         xlim(ax_spec, [0 1]); ylim(ax_spec, [0 1]);
665 
666         ax_freq = axes('Parent', main_panel, 'Units', 'normalized', ...
667             'Position', [0.48 0.55 0.48 0.38], 'Color', [0.09 0.11 0.18]);
668         xlabel(ax_freq, 'Frequency (Hz)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
669         ylabel(ax_freq, 'Amplitude (mm)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
670         title(ax_freq, 'Frequency Response - Sweep Test', 'Color', 'white', 'FontSize', 10, 'FontWeight', 'bold');
671         grid(ax_freq, 'on'); hold(ax_freq, 'on');
672 
673         ax_time = axes('Parent', main_panel, 'Units', 'normalized', ...
674             'Position', [0.48 0.08 0.48 0.38], 'Color', [0.09 0.11 0.18]);
675         xlabel(ax_time, 'Time (s)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
676         ylabel(ax_time, 'Displacement (mm)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
677         title(ax_time, 'Time Domain Response', 'Color', 'white', 'FontSize', 10, 'FontWeight', 'bold');
678         grid(ax_time, 'on'); hold(ax_time, 'on');
679 
680         status_text = uicontrol('Parent', afig, 'Style', 'text', ...
681             'String', sprintf('Vibration testing %s (%.2f x %.2f mm)...', d.sample_name, d.width, d.thickness), ...
682             'Position', [10, 10, 1280, 30], ...
683             'BackgroundColor', [0.1 0.1 0.15], 'ForegroundColor', [0.28 1.0 0.44], ...
684             'FontSize', 10, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');
685 
686         % Vibration parameters
687         fn = d.natural_freq;
688         zeta = d.damping_ratio;
689         sweep_start = 5;
690         sweep_end = 200;
691         sweep_time = 10; % seconds
692         n_frames = 200;
693 
694         % Specimen geometry
695         spec_left = 0.30;
696         spec_right = 0.70;
697         spec_bottom = 0.25;
698         spec_top = 0.75;
699         spec_center_x = (spec_left + spec_right) / 2;
700         spec_height = spec_top - spec_bottom;
701 
702         % Shaker table
703         table_y = spec_bottom - 0.05;
704 
705         % Data storage
706         freq_sweep = linspace(sweep_start, sweep_end, n_frames);
707         amplitude_history = [];
708         freq_history = [];
709 
710         % Create frequency response curve (theoretical)
711         f_response = linspace(0.1, 300, 500);
712         H_response = 1 ./ sqrt((1 - (f_response/fn).^2).^2 + (2*zeta*f_response/fn).^2);
713         H_response = H_response / max(H_response) * 2; % Scale amplitude
714 
715         plot(ax_freq, f_response, H_response, 'g--', 'LineWidth', 1.5, 'Color', [0.3 0.8 0.3]);
716 
717         h_freq_plot = plot(ax_freq, NaN, NaN, 'r-o', 'LineWidth', 2.5, 'MarkerSize', 6);
```

```
718         h_time_plot = plot(ax_time, NaN, NaN, 'b-', 'LineWidth', 2);
719
720         for frame = 1:n_frames
721             if ~ishandle(afig); return; end
722
723             progress = frame / n_frames;
724             current_freq = sweep_start + (sweep_end - sweep_start) * progress;
725
726             % Calculate response amplitude at current frequency
727             r = current_freq / fn;
728             amplification = 1 / sqrt((1 - r^2)^2 + (2*zeta*r)^2);
729             amplitude = amplification * 1.5; % mm
730
731             % Store history
732             freq_history = [freq_history, current_freq];
733             amplitude_history = [amplitude_history, amplitude];
734
735             % Update frequency response plot
736             set(h_freq_plot, 'XData', freq_history, 'YData', amplitude_history);
737             xlim(ax_freq, [0, sweep_end]);
738             ylim(ax_freq, [0, max([amplitude_history, 3])]);
739
740             % Generate time domain signal at current frequency
741             t = linspace(0, 0.2, 200);
742             y_signal = amplitude * sin(2*pi*current_freq*t) .* exp(-zeta*2*pi*current_freq*t);
743             set(h_time_plot, 'XData', t, 'YData', y_signal);
744             xlim(ax_time, [0, 0.2]);
745             ylim(ax_time, [-2.5, 2.5]);
746
747             % Update specimen visualization
748             cla(ax_spec);
749             axis(ax_spec, 'off');
750             hold(ax_spec, 'on');
751             xlim(ax_spec, [0 1]); ylim(ax_spec, [0 1]);
752
753             % Draw shaker table
754             fill(ax_spec, [spec_left-0.15, spec_right+0.15, spec_right+0.15, spec_left-0.15], ...
755                 [table_y-0.03, table_y-0.03, table_y, table_y], ...
756                 [0.35, 0.35, 0.45], 'EdgeColor', [0.5, 0.5, 0.6], 'LineWidth', 2);
757
758             % Draw specimen (showing vibration mode shape)
759             time_vib = frame * 0.05;
760             vib_amplitude = amplitude / 10 * (current_freq / fn);
761             vib_amplitude = min(vib_amplitude, 0.08);
762
763             % Create vibrating shape
764             n_points = 30;
765             x_spec = linspace(spec_left, spec_right, n_points);
766             y_base = linspace(spec_top, spec_bottom, n_points);
767
768             % Mode shape (first bending mode)
769             mode_shape = sin(pi * (x_spec - spec_left) / (spec_right - spec_left));
770             vibration = vib_amplitude * mode_shape * sin(2*pi*current_freq*time_vib);
771
772             % Color based on frequency ratio
773             if r < 0.8
774                 spec_color = [0.2, 0.5, 0.8];
775             elseif r < 1.2
776                 spec_color = [0.9, 0.3, 0.1];
777             else
778                 spec_color = [0.2, 0.7, 0.3];
779             end
780
781             % Draw deformed specimen
782             for i = 1:n_points-1
783                 fill(ax_spec, [x_spec(i), x_spec(i+1), x_spec(i+1), x_spec(i)], ...
784                     [y_base(i)+vibration(i), y_base(i+1)+vibration(i+1), ...
785                      y_base(i+1)-0.02, y_base(i)-0.02], ...
786                     spec_color, 'EdgeColor', spec_color*0.7, 'LineWidth', 1);
787             end
788
789             % Draw vibration arrows
790             for i = 1:5
791                 arrow_x = spec_center_x + (i-3) * 0.08;
792                 arrow_y = spec_top + 0.05 + sin(2*pi*current_freq*time_vib + i) * 0.03;
793                 plot(ax_spec, [arrow_x, arrow_x], [spec_top, arrow_y], 'y-', 'LineWidth', 1);
794                 text(ax_spec, arrow_x, arrow_y+0.01, '↕', 'Color', [0.9, 0.9, 0.5], 'FontSize', 8, 'HorizontalAlignment', 'center');
795             end
796
797             % Labels
798             text(ax_spec, 0.5, spec_top + 0.12, 'VIBRATION INPUT', ...
799                 'Color', [0.9, 0.9, 0.5], 'FontSize', 9, ...
800                 'HorizontalAlignment', 'center', 'FontWeight', 'bold');
801
802             text(ax_spec, 0.5, (spec_top + spec_bottom)/2, ...
803                 sprintf('%s\n%s\nf_n = %.1f Hz\ζn = %.3f\n%.1fx%.1f mm', ...
804                 d.sample_name, d.material_class, fn, zeta, d.width, d.thickness), ...
805                 'Color', 'white', 'FontSize', 7, 'HorizontalAlignment', 'center', ...
806                 'FontWeight', 'bold', 'BackgroundColor', [0, 0, 0, 0.5]);
807
808             % Resonance indicator
```

```
            if abs(r - 1) < 0.1
                text(ax_spec, 0.5, spec_bottom - 0.1, '⚠ RESONANCE ⚠', ...
                    'Color', [1, 0.5, 0.1], 'FontSize', 10, 'FontWeight', 'bold', ...
                    'HorizontalAlignment', 'center');
            end

            % Update status
            status_str = sprintf('VIBRATION | Frame %d/%d | f=%.1f Hz | A=%.2f mm | f_n=%.1f Hz | ζ=%.3f | Dim: %.2fx%.2f mm', ...
                frame, n_frames, current_freq, amplitude, fn, zeta, d.width, d.thickness);
            set(status_text, 'String', status_str);
            pause(0.03);
        end

        if ishandle(afig)
            text(ax_spec, 0.5, 0.5, ...
                sprintf('VIBRATION TEST COMPLETE\nNatural Frequency = %.1f Hz\nDamping Ratio = %.3f\nMax Amplitude = %.2f mm\nDimensions: %.2f x %.2f mm', ...
                fn, zeta, max(amplitude_history), d.width, d.thickness), ...
                'Color', [0.35, 1, 0.45], 'FontSize', 11, 'FontWeight', 'bold', ...
                'HorizontalAlignment', 'center', 'BackgroundColor', [0.04, 0.08, 0.12], ...
                'EdgeColor', [0.2, 0.55, 0.3], 'Margin', 10);
        end
    end

% ========================================================================
%  ANIMATION 2: DROP-WEIGHT IMPACT
% ========================================================================
    function animate_drop_impact(d, app2)
        h_drop = 1.20; mass = 5.50; g_acc = 9.81;
        v0     = sqrt(2*g_acc*h_drop);
        E_in   = mass*g_acc*h_drop;

        E_eff = max(d.young_modulus*1e6, 1e9);
        R_tup = 0.008;
        k_h   = max((4/3)*sqrt(R_tup)*E_eff^0.5, 2e7);

        dt = 2e-6; t_end = 0.025;
        tvec = 0:dt:t_end;
        N_sim = length(tvec);
        zv = zeros(1,N_sim);
        vv = zeros(1,N_sim);
        Fv = zeros(1,N_sim);
        vv(1) = v0;

        for i = 1:N_sim-1
            delta = max(0, zv(i));
            Fc    = min(k_h * delta^1.5, mass*g_acc*50);
            acc   = (mass*g_acc - Fc) / mass;
            vv(i+1) = vv(i) + acc*dt;
            zv(i+1) = zv(i) + vv(i)*dt;
            Fv(i)   = Fc;
            if zv(i+1) < -1e-4 && i > 500
                n_tail = N_sim - i - 1;
                if n_tail > 0
                    decay = exp(-(1:n_tail)*dt*200);
                    vv(i+2:N_sim) = vv(i+1) * decay;
                end
                zv(i+2:N_sim) = 0;
                Fv(i+1:N_sim) = 0;
                break;
            end
        end

        zmm = zv*1000;
        FkN = Fv/1000;
        Eabs = cumtrapz(zmm, FkN);
        Eabs = max(0, Eabs);

        afig = make_anim_fig(sprintf('ASTM D7136 Drop Impact — %s | %s | %s (%.2fx%.2f mm)', ...
            d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness));

        ax_a = axes('Parent',afig,'Color',[0.06 0.08 0.13], ...
            'Units','normalized','Position',[0.02 0.08 0.50 0.82]);
        axis(ax_a,'off'); hold(ax_a,'on');
        xlim(ax_a,[0 1]); ylim(ax_a,[0 1.10]);

        ax_f = make_dark_ax(afig,[0.57 0.57 0.40 0.32],'Displacement (mm)','Force (kN)','Force vs Displacement');
        ax_e = make_dark_ax(afig,[0.57 0.12 0.40 0.32],'Displacement (mm)','Energy (J)','Energy Absorbed');

        hFl = plot(ax_f,NaN,NaN,'Color',[0.15 0.90 0.72],'LineWidth',2.2);
        hEl = plot(ax_e,NaN,NaN,'Color',[0.25 0.92 0.35],'LineWidth',2.2);
        hold(ax_f,'on'); hold(ax_e,'on');
        yline(ax_e,E_in,'--','Color',[0.95 0.28 0.20],'LineWidth',1.8,'Label','E_{in}','LabelHorizontalAlignment','left');

        for rx = [0.44 0.56]
            fill(ax_a,[rx-0.010 rx+0.010 rx+0.010 rx-0.010],[0.13 0.13 0.95 0.95], ...
                [0.22 0.26 0.34],'EdgeColor',[0.35 0.42 0.55],'LineWidth',1.2);
        end

        fill(ax_a,[0.05 0.95 0.95 0.05],[0.036 0.036 0.080 0.080], ...
            [0.25 0.28 0.36],'EdgeColor',[0.45 0.50 0.62],'LineWidth',1.5);

```

```
900         spec_y=0.080; spec_h=0.042;
901         fill(ax_a,[0.12 0.88 0.88 0.12],[spec_y spec_y spec_y+spec_h spec_y+spec_h], ...
902             [0.22 0.44 0.72],'EdgeColor',[0.38 0.60 0.92],'LineWidth',2.0);
903         text(ax_a,0.50,spec_y+spec_h/2,sprintf(' %s [%s] %.1fx%.1fmm', ...
904             d.sample_name,d.material_class,d.width,d.thickness), ...
905             'Color','white','FontSize',8,'HorizontalAlignment','center','FontWeight','bold');
906
907         stat_t = text(ax_a,0.50,1.055,'Releasing impactor...', ...
908             'Color',[1.0 0.88 0.22],'FontSize',10,'FontWeight','bold','HorizontalAlignment','center');
909
910         y_top = 0.950; y_spec_top = spec_y+spec_h;
911         travel_range = y_top - y_spec_top;
912
913         Dd_acc=[]; Fd_acc=[];
914         n_frames = 140;
915
916         for fr = 1:n_frames
917             if ~ishandle(afig); return; end
918             frac  = fr/n_frames;
919             sim_i = max(1, min(round(frac*N_sim), N_sim));
920             z_cur = zv(sim_i); F_cur = FkN(sim_i);
921             v_cur = vv(sim_i); d_mm  = zmm(sim_i);
922
923             frac_fall = min(1, frac*t_end/max(sqrt(2*h_drop/g_acc),1e-9));
924             if z_cur <= 0
925                 y_tup_bot = y_top - frac_fall*travel_range;
926             else
927                 defl_n = min(z_cur/0.012, 1.0);
928                 y_tup_bot = y_spec_top - defl_n*spec_h*0.55;
929             end
930
931             delete(findobj(ax_a,'Tag','impactor'));
932
933             cx=0.50; tw=0.038; ch=0.100;
934             fill(ax_a,[cx-tw cx+tw cx+tw cx-tw], ...
935                 [y_tup_bot y_tup_bot y_tup_bot+ch y_tup_bot+ch], ...
936                 [0.65 0.28 0.08],'EdgeColor',[0.85 0.50 0.15],'LineWidth',2,'Tag','impactor');
937             th_h=linspace(pi,2*pi,60);
938             fill(ax_a, cx+0.030*cos(th_h), y_tup_bot+0.030*abs(sin(th_h)), ...
939                 [0.85 0.42 0.12],'EdgeColor',[0.95 0.62 0.20],'LineWidth',2,'Tag','impactor');
940             text(ax_a,cx,y_tup_bot+ch+0.030, ...
941                 sprintf('%.1fkg | v=%.2fm/s',mass,abs(v_cur)), ...
942                 'Color',[1 0.88 0.35],'FontSize',7.5,'FontWeight','bold', ...
943                 'HorizontalAlignment','center','Tag','impactor');
944
945             delete(findobj(ax_a,'Tag','specdef'));
946             if z_cur > 1e-5
947                 def_d = min(d_mm/20, spec_h*0.70);
948                 n_sp=100; xsp=linspace(0.12,0.88,n_sp);
949                 sigma_g=0.050+def_d/spec_h*0.020;
950                 gauss=exp(-((xsp-cx).^2)/(2*sigma_g^2));
951                 top_y=(spec_y+spec_h)-def_d.*gauss;
952                 dmg = min(def_d/(spec_h*0.55), 1);
953                 sc2 = (1-dmg)*[0.22 0.44 0.72]+dmg*[0.75 0.20 0.06];
954                 fill(ax_a,[xsp,fliplr(xsp)],[top_y,spec_y*ones(1,n_sp)], ...
955                     sc2,'EdgeColor',sc2*0.7+[0.15 0.15 0.15],'LineWidth',1.5,'Tag','specdef');
956                 if F_cur > 0.3*max(FkN)
957                     for ci3=1:round(3+6*dmg)
958                         crx=cx+(rand()-0.5)*0.06*dmg*2;
959                         cry=spec_y+rand()*def_d;
960                         plot(ax_a,[crx crx+(rand()-0.5)*0.02],[cry cry-rand()*def_d*0.7], ...
961                             '-','Color',[1 0.92 0.3+0.4*rand()],'LineWidth',0.8+rand(),'Tag','specdef');
962                     end
963                 end
964             end
965
966             if z_cur > 1e-5
967                 Dd_acc(end+1)=d_mm; Fd_acc(end+1)=F_cur;
968                 set(hFl,'XData',Dd_acc,'YData',Fd_acc);
969                 Eacc = Eabs(1:length(Dd_acc));
970                 set(hEl,'XData',Dd_acc,'YData',Eacc);
971                 if max(Dd_acc)>0
972                     xlim(ax_f,[0,max(Dd_acc)*1.35+0.1]);
973                     xlim(ax_e,[0,max(Dd_acc)*1.35+0.1]);
974                 end
975                 if max(Fd_acc)>0; ylim(ax_f,[0,max(Fd_acc)*1.35+0.05]); end
976                 if max(Eacc)>0;   ylim(ax_e,[0,max(max(Eacc),E_in)*1.25+0.5]); end
977             end
978
979             ph='Free fall';
980             if z_cur>0 && v_cur>0; ph='IMPACT ▼ Loading'; end
981             if z_cur>0 && v_cur<0; ph='IMPACT ▲ Rebound'; end
982             set(stat_t,'String',sprintf('DROP WEIGHT | %s | t=%.2fms | v=%.3fm/s | F=%.3fkN | δ=%.4fmm | Dim: %.1fx%.1f mm', ...
983                 ph,frac*t_end*1000,abs(v_cur),F_cur,d_mm,d.width,d.thickness));
984             drawnow limitrate; pause(0.018);
985         end
986
987         if ishandle(afig)
988             [Fpk,ipk]=max(FkN); dpk=zmm(ipk); Ef=max(Eabs); eff=Ef/E_in*100;
989             set(stat_t,'String',sprintf('DROP WEIGHT COMPLETE | F_pk=%.3fkN | δ=%.4fmm | Eabs=%.2fJ (%.1f%%) | Dim: %.1fx%.1f mm', ...
990                 Fpk,dpk,Ef,eff,d.width,d.thickness),'Color',[0.28 1 0.42]);
```

```
        end
    end

% ======================================================================
%  ANIMATION 3: CHARPY IMPACT
% ======================================================================
    function animate_charpy(d, app2)
        E_charpy = 0.5 * d.thickness * d.width * (d.uts/1e3)^2 / d.young_modulus * 1e6;
        E_charpy = max(5, min(E_charpy, 150));

        afig = make_anim_fig(sprintf('Charpy Impact ASTM E23 — %s | %s | %s (%.2fx%.2f mm)', ...
            d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness));
        ax_a = axes('Parent',afig,'Color',[0.06 0.08 0.13], ...
            'Units','normalized','Position',[0.02 0.06 0.60 0.86]);
        axis(ax_a,'off'); hold(ax_a,'on');
        xlim(ax_a,[0 1]); ylim(ax_a,[0 1]);

        ax_p = make_dark_ax(afig,[0.66 0.35 0.32 0.54],'Swing Angle (°)','Energy (J)','Charpy Energy Absorbed');
        stat_t = text(ax_a,0.50,0.96,'Charpy Pendulum Test','Color',[1 0.88 0.22], ...
            'FontSize',10,'FontWeight','bold','HorizontalAlignment','center');

        fill(ax_a,[0.18 0.82 0.82 0.18],[0.12 0.12 0.18 0.18],[0.30 0.30 0.35], ...
            'EdgeColor',[0.55 0.55 0.65],'LineWidth',2);
        fill(ax_a,[0.38 0.62 0.62 0.38],[0.18 0.18 0.22 0.22],[0.22 0.44 0.72], ...
            'EdgeColor',[0.40 0.62 0.92],'LineWidth',2);
        text(ax_a,0.50,0.20,sprintf('%s\n%.1fx%.1f mm', d.sample_name, d.width, d.thickness), ...
            'Color','white','FontSize',8,'HorizontalAlignment','center','FontWeight','bold');

        pivot_x=0.50; pivot_y=0.88; arm_len=0.62;
        plot(ax_a,pivot_x,pivot_y,'wo','MarkerSize',12,'LineWidth',2,'MarkerFaceColor',[0.6 0.6 0.7]);

        ang_init = -145; ang_final = 75;
        impact_ang = -5;
        hl_e = plot(ax_p,NaN,NaN,'Color',[0.15 0.85 0.70],'LineWidth',2.2);
        grid(ax_p,'on');
        ang_data=[]; E_data=[];

        n_frames=120;
        for fr=1:n_frames
            if ~ishandle(afig); return; end
            frac=fr/n_frames;

            if frac < 0.45
                ang_now = ang_init + frac/0.45*(impact_ang-ang_init);
                phase='Swing down';
            elseif frac < 0.55
                ang_now = impact_ang;
                phase='IMPACT';
            else
                ang_now = impact_ang + (frac-0.55)/0.45*(ang_final-impact_ang);
                phase='Swing up';
            end

            delete(findobj(ax_a,'Tag','pendulum'));
            ang_rad = ang_now*pi/180;
            bob_x = pivot_x + arm_len*sin(ang_rad);
            bob_y = pivot_y - arm_len*cos(ang_rad);
            line(ax_a,[pivot_x bob_x],[pivot_y bob_y],'Color',[0.60 0.65 0.75],'LineWidth',3,'Tag','pendulum');
            fill(ax_a,bob_x+0.04*[-1 1 0.5 -0.5], bob_y+0.04*[0 0 -1 -1], ...
                [0.75 0.50 0.15],'EdgeColor',[0.90 0.70 0.25],'LineWidth',2,'Tag','pendulum');

            E_cur = E_charpy * max(0, (frac-0.50)/0.50);
            if frac > 0.50
                ang_data(end+1)=ang_now; E_data(end+1)=E_cur;
                set(hl_e,'XData',ang_data,'YData',E_data);
                if max(ang_data)>min(ang_data)
                    xlim(ax_p,[min(ang_data)-5, max(ang_data)+5]);
                end
                ylim(ax_p,[0, E_charpy*1.30+1]);
            end

            set(stat_t,'String',sprintf('CHARPY | %s | Angle=%.1f° | Energy=%.2f J | Specimen: %.1fx%.1f mm', ...
                phase, ang_now, E_cur, d.width, d.thickness));
            drawnow limitrate; pause(0.022);
        end

        if ishandle(afig)
            text(ax_a,0.50,0.50,sprintf('CHARPY COMPLETE\nImpact Energy = %.2f J\nStrength = %.1f MPa\nDim: %.1fx%.1f mm', ...
                E_charpy, d.uts, d.width, d.thickness),'Color',[0.35 1 0.45],'FontSize',11,'FontWeight','bold', ...
                'HorizontalAlignment','center','BackgroundColor',[0.04 0.08 0.12], ...
                'EdgeColor',[0.20 0.55 0.30],'Margin',8);
        end
    end

% ======================================================================
%  ANIMATION 4: IZOD IMPACT
% ======================================================================
    function animate_izod(d, app2)
        E_izod = 0.40 * d.thickness * d.width * (d.uts/1e3)^2 / d.young_modulus * 1e6;
        E_izod = max(2, min(E_izod, 80));

```

```
        afig = make_anim_fig(sprintf('Izod Impact ASTM D256 — %s | %s | %s (%.2fx%.2f mm)', ...
            d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness));
        ax_a = axes('Parent',afig,'Color',[0.06 0.08 0.13], ...
            'Units','normalized','Position',[0.02 0.06 0.58 0.86]);
        axis(ax_a,'off'); hold(ax_a,'on');
        xlim(ax_a,[0 1]); ylim(ax_a,[0 1]);

        ax_p = make_dark_ax(afig,[0.64 0.35 0.33 0.54],'Swing Angle (°)','Energy (J)','Izod Energy Absorbed');
        stat_t = text(ax_a,0.50,0.96,'Izod Pendulum Test','Color',[1 0.88 0.22], ...
            'FontSize',10,'FontWeight','bold','HorizontalAlignment','center');

        fill(ax_a,[0.40 0.60 0.60 0.40],[0.10 0.10 0.40 0.40],[0.30 0.30 0.35], ...
            'EdgeColor',[0.55 0.55 0.65],'LineWidth',2);
        fill(ax_a,[0.46 0.54 0.54 0.46],[0.40 0.40 0.65 0.65],[0.22 0.44 0.72], ...
            'EdgeColor',[0.40 0.62 0.92],'LineWidth',2.5);
        text(ax_a,0.50,0.52,sprintf('%s\n%.1fx%.1f mm', d.sample_name, d.width, d.thickness), ...
            'Color','white','FontSize',8,'HorizontalAlignment','center','FontWeight','bold');

        pivot_x=0.08; pivot_y=0.55; arm_len=0.48;
        plot(ax_a,pivot_x,pivot_y,'wo','MarkerSize',12,'LineWidth',2,'MarkerFaceColor',[0.6 0.6 0.7]);

        ang_init=-80; ang_final=55; impact_ang=0;
        hl_e=plot(ax_p,NaN,NaN,'Color',[0.90 0.50 0.15],'LineWidth',2.2);
        grid(ax_p,'on'); ang_data2=[]; E_data2=[];

        n_frames=120;
        for fr=1:n_frames
            if ~ishandle(afig); return; end
            frac=fr/n_frames;
            if frac<0.40;      ang_now=ang_init+frac/0.40*(impact_ang-ang_init); ph='Swing down';
            elseif frac<0.50;  ang_now=impact_ang; ph='IMPACT';
            else;              ang_now=impact_ang+(frac-0.50)/0.50*(ang_final-impact_ang); ph='Swing up'; end

            delete(findobj(ax_a,'Tag','pend2'));
            ar=ang_now*pi/180;
            bx=pivot_x+arm_len*cos(ar); by=pivot_y+arm_len*sin(ar);
            line(ax_a,[pivot_x bx],[pivot_y by],'Color',[0.60 0.65 0.75],'LineWidth',3,'Tag','pend2');
            fill(ax_a,bx+0.04*[0 1 1 0], by+0.04*[-0.5 -0.5 0.5 0.5], ...
                [0.75 0.50 0.15],'EdgeColor',[0.90 0.70 0.25],'LineWidth',2,'Tag','pend2');

            E_cur=E_izod*max(0,(frac-0.48)/0.52);
            if frac>0.48
                ang_data2(end+1)=ang_now; E_data2(end+1)=E_cur;
                set(hl_e,'XData',ang_data2,'YData',E_data2);
                if max(ang_data2)>min(ang_data2); xlim(ax_p,[min(ang_data2)-5, max(ang_data2)+5]); end
                ylim(ax_p,[0,E_izod*1.30+1]);
            end
            set(stat_t,'String',sprintf('IZOD | %s | Angle=%.1f° | Energy=%.2f J | Specimen: %.1fx%.1f mm', ...
                ph,ang_now,E_cur,d.width,d.thickness));
            drawnow limitrate; pause(0.022);
        end
        if ishandle(afig)
            text(ax_a,0.50,0.80,sprintf('IZOD COMPLETE\nEnergy = %.2f J\nUTS = %.1f MPa\nDim: %.1fx%.1f mm', ...
                E_izod,d.uts,d.width,d.thickness),'Color',[0.35 1 0.45],'FontSize',11,'FontWeight','bold', ...
                'HorizontalAlignment','center','BackgroundColor',[0.04 0.08 0.12], ...
                'EdgeColor',[0.20 0.55 0.30],'Margin',8);
        end
    end

% ========================================================================
%  ANIMATION 5: FLEXURAL / 3-POINT BEND
% ========================================================================
    function animate_flexural(d, app2)
        L_span = 100; % mm
        b = d.width; h = d.thickness;
        E_flex = d.young_modulus;
        sigma_f = 1.50 * d.uts;
        F_flex  = 2 * sigma_f * b * h^2 / (3 * L_span);
        F_flex  = F_flex / 1000;
        delta_max = (F_flex*1000 * L_span^3) / (48 * E_flex * b*h^3/12) * 1e3;
        delta_max = max(0.5, min(delta_max, 15));

        afig = make_anim_fig(sprintf('Flexural Test ASTM D790 — %s | %s | %s (%.2fx%.2f mm)', ...
            d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness));
        ax_a = axes('Parent',afig,'Color',[0.06 0.08 0.13], ...
            'Units','normalized','Position',[0.02 0.08 0.52 0.84]);
        axis(ax_a,'off'); hold(ax_a,'on');
        xlim(ax_a,[0 1]); ylim(ax_a,[0 1]);

        ax_fd = make_dark_ax(afig,[0.58 0.57 0.39 0.32],'Deflection (mm)','Load (kN)','Flexural Load-Deflection');
        ax_fs = make_dark_ax(afig,[0.58 0.12 0.39 0.32],'Strain (%)','Flex. Stress (MPa)','Flexural Stress-Strain');

        hl_fd = plot(ax_fd,NaN,NaN,'Color',[0.15 0.85 0.70],'LineWidth',2.2);
        hl_fs = plot(ax_fs,NaN,NaN,'Color',[0.90 0.50 0.15],'LineWidth',2.2);
        grid(ax_fd,'on'); grid(ax_fs,'on');

        stat_t = text(ax_a,0.50,0.96,'3-Point Flexural Test', ...
            'Color',[1 0.88 0.22],'FontSize',10,'FontWeight','bold','HorizontalAlignment','center');

        for sx=[0.12 0.88]
            fill(ax_a,[sx-0.03 sx+0.03 sx],[0.30 0.30 0.37],[0.45 0.45 0.55], ...
```

```
            'EdgeColor',[0.65 0.65 0.75],'LineWidth',1.5);
        end

        def_data=[]; load_data=[]; str_data=[]; fstress_data=[];
        n_frames=120;
        for fr=1:n_frames
            if ~ishandle(afig); return; end
            frac=fr/n_frames;
            def_cur = frac^1.1 * delta_max;
            F_cur   = frac^0.9 * F_flex;
            str_cur = frac * 6*h*def_cur/(L_span^2) * 100;
            fstr_cur= frac * sigma_f;

            delete(findobj(ax_a,'Tag','beam'));
            n_b=80; xb=linspace(0.12,0.88,n_b);
            sag = def_cur/delta_max * 0.08 * sin(pi*(xb-0.12)/0.76);
            yb  = 0.38 - sag;
            dmg = frac;
            bc  = (1-dmg)*[0.22 0.44 0.72]+dmg*[0.75 0.20 0.06];
            fill(ax_a,[xb,fliplr(xb)],[yb,yb-0.04*ones(1,n_b)], ...
                bc,'EdgeColor',bc*0.7,'LineWidth',1.8,'Tag','beam');

            ind_y = 0.38 + 0.06 - frac*0.06 - def_cur/delta_max*0.04;
            fill(ax_a,[0.47 0.53 0.53 0.47],[ind_y+0.08 ind_y+0.08 ind_y ind_y], ...
                [0.65 0.28 0.08],'EdgeColor',[0.85 0.50 0.15],'LineWidth',2,'Tag','beam');
            th2=linspace(pi,2*pi,40);
            fill(ax_a,0.50+0.026*cos(th2),ind_y+0.026*abs(sin(th2)), ...
                [0.85 0.42 0.12],'EdgeColor',[0.95 0.62 0.20],'Tag','beam');
            text(ax_a,0.50,ind_y+0.12,sprintf('F=%.3f kN',F_cur), ...
                'Color',[1 0.88 0.35],'FontSize',8,'HorizontalAlignment','center','FontWeight','bold','Tag','beam');

            if fr>1
                def_data(end+1)=def_cur; load_data(end+1)=F_cur;
                str_data(end+1)=str_cur; fstress_data(end+1)=fstr_cur;
                set(hl_fd,'XData',def_data,'YData',load_data);
                set(hl_fs,'XData',str_data,'YData',fstress_data);
                if max(def_data)>0; xlim(ax_fd,[0,max(def_data)*1.30+0.1]); ylim(ax_fd,[0,max(load_data)*1.30+0.01]); end
                if max(str_data)>0; xlim(ax_fs,[0,max(str_data)*1.30+0.01]); ylim(ax_fs,[0,max(fstress_data)*1.30+1]); end
            end

            set(stat_t,'String',sprintf('FLEXURAL | Frame %d/%d | δ=%.2fmm | F=%.3fkN | σ_f=%.1fMPa | Specimen: %.1fx%.1f mm', ...
                fr,n_frames,def_cur,F_cur,fstr_cur,d.width,d.thickness));
            drawnow limitrate; pause(0.025);
        end
        if ishandle(afig)
            text(ax_a,0.50,0.65,sprintf('FLEXURAL COMPLETE\nFlex. Strength = %.1f MPa\nMax Load = %.3f kN\nMax Deflection = %.2f mm\nDim: %.1fx%.1f mm', ...
                sigma_f,F_flex,delta_max,d.width,d.thickness),'Color',[0.35 1 0.45],'FontSize',10,'FontWeight','bold', ...
                'HorizontalAlignment','center','BackgroundColor',[0.04 0.08 0.12], ...
                'EdgeColor',[0.20 0.55 0.30],'Margin',8);
        end
    end

% ========================================================================
%  ANIMATION 6: COMPRESSIVE TEST
% ========================================================================
    function animate_compressive(d, app2)
        [sv, tv] = make_compressive_curve(d);

        afig = figure('Name', sprintf('Compressive Test ASTM D695 — %s | %s | %s (%.2fx%.2f mm)', ...
            d.sample_name, app2.tester_name, app2.company_name, d.width, d.thickness), ...
            'NumberTitle', 'off', 'Position', [50 50 1300 800], ...
            'Color', [0.06 0.08 0.13], 'MenuBar', 'none', 'ToolBar', 'none');

        main_panel = uipanel('Parent', afig, 'Units', 'normalized', ...
            'Position', [0.02 0.05 0.96 0.90], 'BackgroundColor', [0.06 0.08 0.13], ...
            'BorderType', 'none');

        ax_spec = axes('Parent', main_panel, 'Units', 'normalized', ...
            'Position', [0.03 0.10 0.40 0.80], 'Color', [0.08 0.10 0.15]);
        axis(ax_spec, 'off'); hold(ax_spec, 'on');
        xlim(ax_spec, [0 1]); ylim(ax_spec, [0 1]);

        ax_ss_live = axes('Parent', main_panel, 'Units', 'normalized', ...
            'Position', [0.48 0.55 0.48 0.38], 'Color', [0.09 0.11 0.18]);
        xlabel(ax_ss_live, 'Compressive Strain (%)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        ylabel(ax_ss_live, 'Compressive Stress (MPa)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        title(ax_ss_live, 'Compressive Stress-Strain Curve', 'Color', 'white', 'FontSize', 10, 'FontWeight', 'bold');
        grid(ax_ss_live, 'on'); hold(ax_ss_live, 'on');

        ax_ld_live = axes('Parent', main_panel, 'Units', 'normalized', ...
            'Position', [0.48 0.08 0.48 0.38], 'Color', [0.09 0.11 0.18]);
        xlabel(ax_ld_live, 'Displacement (mm)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        ylabel(ax_ld_live, 'Compressive Load (kN)', 'Color', [0.7 0.82 0.95], 'FontSize', 9);
        title(ax_ld_live, 'Load vs Displacement', 'Color', 'white', 'FontSize', 10, 'FontWeight', 'bold');
        grid(ax_ld_live, 'on'); hold(ax_ld_live, 'on');

        h_ss = plot(ax_ss_live, NaN, NaN, 'r-', 'LineWidth', 2.5);
        h_ld = plot(ax_ld_live, NaN, NaN, 'b-', 'LineWidth', 2.5);

        status_text = uicontrol('Parent', afig, 'Style', 'text', ...
            'String', sprintf('Compressive testing %s (%.2f x %.2f mm)...', d.sample_name, d.width, d.thickness), ...
```

```
            'Position', [10, 10, 1280, 30], ...
            'BackgroundColor', [0.1 0.1 0.15], 'ForegroundColor', [0.28 1.0 0.44], ...
            'FontSize', 10, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');

        dv = sv .* d.gauge_length / 100;
        lv = tv .* d.area / 1e3;

        spec_left = 0.30;
        spec_right = 0.70;
        spec_bottom = 0.20;
        spec_top = 0.80;
        spec_center = (spec_left + spec_right) / 2;

        n_frames = 150;
        failure_frame = round(n_frames * 0.85);

        acc_strain = [];
        acc_stress = [];
        acc_disp = [];
        acc_load = [];

        for frame = 1:n_frames
            if ~ishandle(afig); return; end
```